# Deep Reinforcement Learning-Based Task Offloading for Vehicular Edge Computing With Flexible RSU-RSU Cooperation

Wenhao Fan, *Member, IEEE*, Yaoyin Zhang, Guangtao Zhou, and Yuan'an Liu

***Abstract*— Vehicle edge computing (VEC) acts as an enhancement to provide low latency and low energy consumption for internet of vehicles (IoV) applications. Mobility of vehicles and load difference of roadside units (RSUs) are two important issues in VEC. The former results in task result reception failures owing to vehicles moving out of the coverage of their current RSUs; the latter leads to system performance degradation owing to load imbalance among the RSUs. They can be well solved by exploiting flexible RSU-RSU cooperation, which has not been fully studied by existing works. In this paper, we propose a novel resource management scheme for joint task offloading, computing resource allocation for vehicles and RSUs, vehicle-to-RSU transmit power allocation, and RSU-to-RSU transmission rate allocation. In our scheme, a task result can be transferred to the RSU where the vehicle is currently located, and a task can be further offloaded from a high-load RSU to a low-load RSU. To minimize the total task processing delay and energy consumption of all the vehicles, we design a twin delayed deep deterministic policy gradient (TD3)-based deep reinforcement learning (DRL) algorithm, where we embed an optimization subroutine to solve 2 sub-problems via numerical methods, thus reducing the training complexity of the algorithm. Extensive simulations are conducted in 6 different scenarios. Compared with 4 reference schemes, our scheme can reduce the total task processing cost by 17.3%-28.4%.**

***Index Terms*— Edge computing, internet of vehicles, vehicle mobility, resource management, task offloading.**

## I. Introduction

VEHICLE edge computing (VEC) [1] is gaining more and more attention owing to the fast development of intelligent vehicles. VEC is the edge computing technology in internet of vehicles (IoV) scenarios. It enhances the performance of computation-intensive and delay-sensitive IoV applications, such as autopilot, in-vehicle intelligence, etc., by offloading the tasks from vehicles to the edge servers (ESs) equipped on or near road site units (RSUs). The essence of VEC is to offload the tasks at the cost of a much lower task transmission delay than cloud computing, and then process the tasks with much more computing resources than the vehicles.

The mobility of vehicles is the most typical characteristic of VEC that differs from the other ES scenarios. Task offloading service for a vehicle needs to consider the holding time [2] that the vehicle keeps in the coverage of the current RSU. The vehicle will suffer from task result reception failure if it runs out of coverage before the RSU sends the result back to the vehicle, further resulting in service interruption.

Another characteristic of VEC is the uneven temporospatial distribution of vehicles [3]. Load difference of RSUs will be caused when multiple geographically hot areas (e.g., traffic jam areas containing a large number of vehicles) and cold areas (e.g., side streets containing several vehicles only) coexist in the topology of the scenario. The load imbalance problem leads to the performance degradation of high-load RSUs and the resource waste of low-load RSUs.

Resource management is vital to fulfilling high-efficiency VEC. It makes task offloading decisions for all the vehicles and allocates the communication and computing resources used in task offloading processes. In multi-RSU VEC scenarios, traditional schemes managed the resources of each RSU separately. In these schemes, a task offloaded to an RSU must be completed during the holding time of the vehicle, and high-load RSUs have to make tasks processed on the vehicles locally to alleviate their loads. Such a pattern restricts the system performance improvement. It also can not fully avoid task offloading service interruption and solve the load imbalance problem among RSUs.

The high-speed interconnections among multiple RSUs can be fully utilized to enable cooperation among the ESs of the RSUs, called RSU-RSU cooperation. By fully exploiting the cooperation, 1) a task result completed on an RSU can be further transmitted to another RSU, and then to the vehicle under its coverage, thus avoiding the above task result reception failure problem; 2) a task offloaded to a high-load RSU can be further transmitted to another low-load RSU and processed on it, thus realizing load balance among multiple RSUs.

Manuscript received 22 April 2023; revised 17 October 2023; accepted 27 December 2023. Date of publication 16 January 2024; date of current version 2 July 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62271086 and Grant 61821001; in part by the Fund of State Key Laboratory of Information Photonics and Optical Communications (Beijing University of Posts and Telecommunications), China; and in part by the Fundamental Research Funds for the Central Universities under Grant 2023PY10. The Associate Editor for this article was H. Jiang. *(Corresponding author: Wenhao Fan.)*

Wenhao Fan is with the School of Electronic Engineering, the Beijing Key Laboratory of Work Safety Intelligent Monitoring, and the National Engineering Research Center for Disaster Backup and Recovery, Beijing University of Posts and Telecommunications (BUPT), Beijing 100876, China (e-mail: whfan@bupt.edu.cn).

Yaoyin Zhang and Yuan'an Liu are with the School of Electronic Engineering and the Beijing Key Laboratory of Work Safety Intelligent Monitoring, Beijing University of Posts and Telecommunications (BUPT), Beijing 100876, China.

Guangtao Zhou is with China Unicom Smart Connection Technology Company Ltd., Beijing 100033, China.

Digital Object Identifier 10.1109/TITS.2024.3349546

1558-0016 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

However, the RSU-RSU cooperation studied by existing works lacked flexibility: 1) The task of a vehicle can be offloaded to the original RSU only, but the task result can be transmitted to the RSU covering the vehicle currently [4], [5], or 2) the task can be offloaded to any RSU properly, but the task result must be sent back to the original RSU, then to the vehicle if it is still under the coverage of the RSU [6], [7], [8]. Both of them restrict further improvements in the system performance.

To this end, in this paper, we propose a deep reinforcement learning (DRL)-based task offloading scheme for a multi-vehicle and multi-RSU VEC scenario with flexible RSU-RSU cooperation. The contributions of our work are as follows:

1) Considering both the task result reception failure of the vehicles and the load imbalance among the RSUs, we leverage the flexible RSU-RSU cooperation to jointly schedule the tasks and task results in the task offloading processes. Different from the existing works that lacked flexibility on task offloading and task result transmission, in our scheme, a task can be offloaded to a proper RSU and the task result can be transmitted to the RSU covering the vehicle currently. We formulate a joint task offloading, computing resource allocation for vehicles and RSUs, vehicle-to-RSU (V2R) transmit power allocation, and RSU-to-RSU (R2R) transmission rate allocation optimization problem to minimize the total task processing delay and energy consumption of all the vehicles in the scenario.

2) To efficiently solve the mixed integer nonlinear programming (MINLP) optimization problem, we design a twin delayed deep deterministic policy gradient (TD3)-based DRL algorithm. Different from the existing works that used DRL or numerical methods to solve the entire problem, in our algorithm, to reduce the complexity of the training process, we embed an optimization subroutine to solve the computing resource allocation sub-problem and the transmit power allocation sub-problem for the corresponding vehicles by using numerical methods, then let the TD3 part of our algorithm handle the other part of the optimization problem, thus decreasing the searching space of the TD3 model. Compared with the approaches that completely rely on the DRL algorithms to solve the optimization problem, our algorithm can provide faster convergence speed and better solution, and compared with the approaches that solve the optimization problem via numerical methods only, our algorithm can provide faster problem-solving speed.

3) We evaluate the convergence of our algorithm concerning the training episodes. Extensive simulations are conducted by varying 6 different crucial parameters. The simulation results demonstrate the superiority of our schemes in comparison with 5 reference schemes.

The rest of our paper is organized as follows: Section II discusses the related works. Section III presents the system model and formulates the optimization problem. Section IV describes the optimization algorithm design. Section V analyzes the simulation results and evaluates the performance of our scheme by comparisons. Finally, Section VI concludes our work.

## II. Related Works

VEC refers to the edge computing applied to IoV scenarios. It uses the computing devices around the vehicle such as other vehicles and ESs of RSUs for data processing and analysis to improve the performance of the vehicle [14]. Resource management plays a key role in VEC systems. It optimizes system performance by efficiently scheduling task offloading and allocating system resources while meeting the QoS requirements of different users. Many existing works have been well done in this area, aiming at minimizing task processing delay [1], [2], [3], [15], [16], [17], [18], system energy consumption [19], [20], [21], system cost [22], [23], [24], [25], [26], etc.

Due to the mobility of vehicles and the limited wireless coverage of RSUs, a vehicle that offloads its task to its affiliated RSU may leave the RSU before the task processing is completed on the ES of the RSU [27]. In this case, the task result cannot be directly returned to the vehicle from the RSU, and then causing a task result reception failure. To this end, some works made decisions that the task must be locally processed in this case [9], [10]. They restricted the flexibility of task offloading and hindered the system performance improvements. On the other side, task result reception failure can be solved by leveraging vehicle-to-vehicle (V2V) communications [11], [18], [28]. The task result can be forwarded by other vehicles and then to the target vehicle, although currently, it is not under the coverage of the RSU. However, this process needs to transmit task data among multiple vehicles, so privacy and security issues will be involved, resulting in such V2V task offloading becoming less practical.

The uneven temporospatial distribution of vehicles in a VEC scenario with multiple RSUs causes load difference of RSUs. In some areas, such as crossroads, markets, stations, etc., the RSUs cover a large number of vehicles with abundant task offloading requests, causing task processing performance degradation due to the high loads of these RSUs; in some areas, such as residential districts, pathways, etc., fewer vehicles are covered by the RSUs, leading to resource wastes owing to low loads of these RSUs. Some works only considered single-RSU scenarios [12], [29]; some works focused on multi-RSU scenarios [13], [30], but they still managed each RSU separately, so the load imbalance problem among RSUs was left unhandled.

RSU-RSU cooperation is the VEC-version of edge-edge cooperation. It is judged as a promising solution to the task result reception failure problem and the load imbalance problem among RSUs. By leveraging the high-speed interconnections among multiple RSUs, a task offloaded to an RSU can be further transferred to another RSU and processed on it, and also a task result completed on an RSU can be further transmitted to another RSU, and then returned to the vehicle under its coverage. Thus, enabled by RSU-RSU cooperation, task result transmission [4], [5] and load transfer [6], [7], [8] can solve the above problems. However, the existing works in this area still lacked flexibility on RSU-RSU cooperation. In [4], [5], and [25], the task of a vehicle can be offloaded to

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DIFFERENCE BETWEEN OUR SCHEME AND OTHER MAIN RELATED SCHEMES

| References | RSU-RSU Cooperation | R2R Task Transmission | R2R Task Result Transmission | Computing Resource Allocation of Vehicles | Computing Resource Allocation of RSUs | R2R Transmission Rate Allocation | V2R Transmit Power Allocation |
|---|---|---|---|---|---|---|---|
| [9] | × | × | × | × | √ | × | × |
| [10] | × | × | × | × | × | × | × |
| [11] | × | × | × | × | × | × | × |
| [12] | × | × | × | × | × | × | × |
| [13] | × | × | × | × | √ | × | × |
| [6] | √ | √ | × | × | √ | √ | × |
| [7] | √ | √ | × | × | √ | × | × |
| [8] | √ | √ | × | × | × | × | × |
| [4] | √ | × | √ | × | × | × | × |
| [5] | √ | × | √ | √ | √ | × | × |
| Our Scheme | √ | √ | √ | √ | √ | √ | √ |

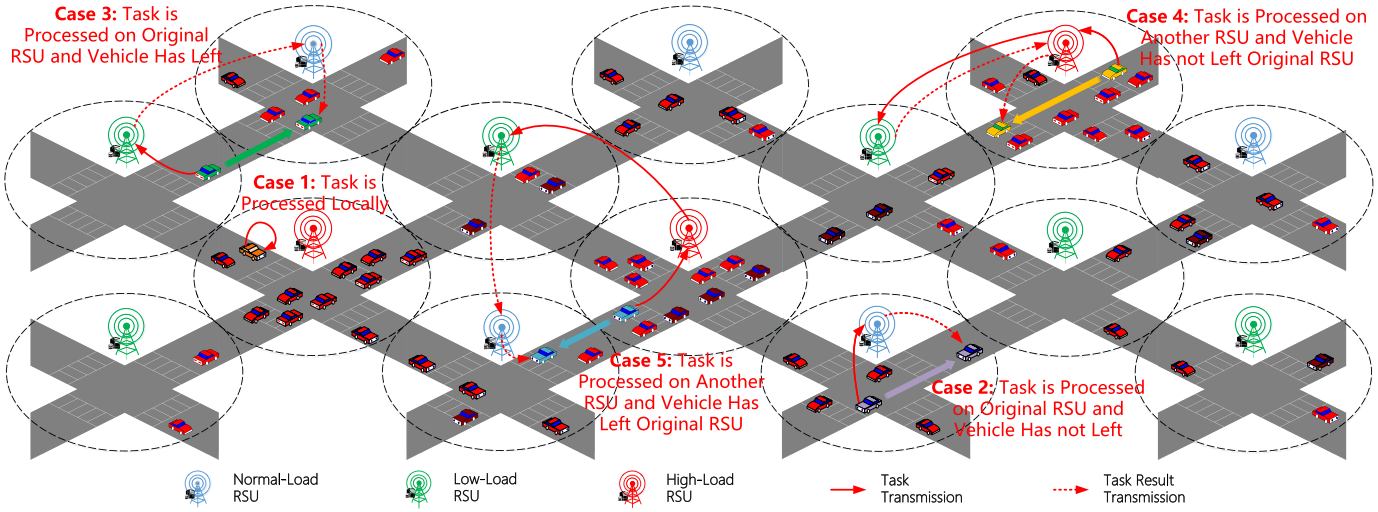



Fig. 1. An RSU-RSU cooperative VEC scenario consisting of 5 task processing cases.

the original RSU only, but the task result can be transmitted to the RSU covering the vehicle currently; in [6], [7], and [8], the task can be offloaded to any RSU properly, but the task result must be sent back to the original RSU, then to the vehicle if it is still under the coverage of the RSU. The other works [17], [25], [26] considered task offloading among multiple RSUs, but they ignored the task result transmission. The lack of flexibility in RSU-RSU cooperation restricts further improvements in the system performance.

In this paper, we provide flexible RSU-RSU cooperation to jointly schedule the tasks and task results in the task offloading processes. Thus, both the task result reception failure problem and the load imbalance problem among RSUs can be solved by offloading tasks to proper RSUs and transmitting task results to the RSUs covering the target vehicles currently. To manage the resources involved in the entire task offloading process, we formulate an optimization problem for joint task offloading, computing resource allocation for vehicles and RSUs, V2R transmit power allocation, and R2R transmission rate allocation. Different from the existing works that optimized latency and energy separately, our goal is to minimize the total task processing delay and energy consumption of all the vehicles in the scenario. We compare our scheme with the main related schemes in Table I.

## III. SYSTEM MODEL

The main symbols used in the system model is listed in Table II.

### A. Network Model

As shown in Fig. 1, we consider an urban topology covered by $M$ RSUs, which are interconnected and are included in a set $\mathcal{M} = \{1, 2, \ldots, M\}$. An RSU $j \in \mathcal{M}$ is equipped with an ES. There $N_j$ vehicles initially under the coverage of RSU $j$ and they are collected by a set $\mathcal{N}_j = \{1, 2, \ldots, N_j\}$. We use vehicle $ij$ to represent the $i$th vehicle ($i \in \mathcal{N}_j$) covered by RSU $j$.

We profile the task of vehicle $ij$ as a 3-tuple vector $< c_{ij}, d_{ij}^{in}, d_{ij}^{out} >$. $c_{ij}$ (in terms of CPU cycles) is the amount of computation required for task processing, $d_{ij}^{in}$ (in terms of bits) is the data size needed for task transmission, and $d_{ij}^{out}$ (in terms of bits) is the data size needed for task result transmission.

To avoid verbosity, we use task $ij$ to represent the task of vehicle $ij$ and use task result $ij$ to represent the task result of vehicle $ij$.

Our scheme has two options to handle task $ij$: local task processing and task offloading. We use $\alpha_{ij} \in \{0, 1\}$ to represent the local task processing decision of vehicle $ij$.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MAIN SYMBOLS USED IN SYSTEM MODEL

| Name | Meaning |
|---|---|
| $M, \mathcal{M}$ | number and set of RSUs |
| $N_j, \mathcal{N}_j$ | number and set of the vehicles originally covered the $j$th RSU |
| $c_{ij}$ | amount of computation of task $ij$ |
| $d_{ij}^{in}$ | data size of task $ij$ |
| $d_{ij}^{out}$ | data size of task result $ij$ |
| $\alpha_{ij}$ | local task processing indicator of vehicle $ij$ |
| $\mathcal{A}$ | set of all local task processing indicators |
| $\beta_{ijkl}$ | task offloading indicator of vehicle $ij$ |
| $\mathcal{B}$ | set of all task offloading indicators |
| $\xi_{ij}$ | index of the next RSU of vehicle $ij$ |
| $\chi_{ij}$ | holding time of vehicle $ij$ |
| $\mathcal{L}_{ij}$ | set of the value range of index $l$ in $\beta_{ijkl}$ |
| $F_{ij}^{v}$ | total computing resource of vehicle $ij$ |
| $f_{ij}^{v}$ | computing resource allocated from $F_{ij}^{v}$ for local task computing |
| $t_{ij}^{v}$ | local task computing delay of vehicle $v$ |
| $\mathcal{F}^{v}$ | set of the local computing resource allocation indicators of all the vehicles |
| $F_{k}^{r}$ | total computing resource of RSU $k$ |
| $f_{ijk}^{r}$ | computing resource allocated from $F_{k}^{r}$ for computing task $ij$ |
| $\mathcal{F}^{r}$ | set of the computing resource allocation indicators of all the RSUs |
| $W$ | channel bandwidth |
| $N_0$ | noise power spectral density |
| $H_{ij}^{v2r}, H_{ijl}^{r2v}$ | channel gains of V2R and R2V transmission for vehicle $ij$ |
| $I_{ij}^{v2r}, I_{ijl}^{r2v}$ | co-channel interference of V2R and R2V transmission for vehicle $ij$ |
| $P_{ij}^{v2r}$ | maximum transmit power of vehicle $ij$ |
| $p_{ij}^{v2r}$ | transmit power allocated for vehicle $ij$ transmitting its task |
| $\mathcal{P}$ | set of transmit power allocation indicators of all the vehicles |
| $\Gamma_{ij}^{v2r}, \Gamma_{ijl}^{r2v}$ | SINR of V2R and R2V transmission for vehicle $ij$ |
| $r_{ij}^{v2r}, r_{ijl}^{r2v}$ | V2R and R2V transmission rates for vehicle $ij$ |
| $t_{ij}^{v2r}, t_{ijl}^{r2v}$ | V2R and R2V transmission delays for vehicle $ij$ |
| $R_{j}^{r2r}$ | total wired transmission rate from RSU $j$ to the other RSUs |
| $r_{jk}^{r2r}$ | transmission rate allocated from $R_{j}^{r2r}$ for the connection from RSU $j$ to RSU $k$ |
| $t_{ijk}^{in}$ | R2R transmission delay for transmitting task $ij$ from RSU $j$ to RSU $k$ |
| $t_{ijkl}^{in}$ | R2R transmission delay for transmitting task result $ij$ from RSU $k$ to RSU $l$ |
| $\mathcal{R}$ | set of all the transmission rate allocation indicators |
| $e_{ij}^{v}$ | energy consumption for local task processing of task $ij$ |
| $\kappa_{ij}$ | constant indicating the effective switched capacitance |
| $e_{ij}^{v2r}$ | energy consumption for V2R task transmission of task $ij$ |
| $w_{ij}$ | weight factor for the delay-energy trade-off of vehicle $ij$ |
| $u$ | total task processing cost of all the vehicles |

$\alpha_{ij} = 1$ means that task $ij$ is chosen to be processed on the vehicle locally, otherwise, $\alpha_{ij} = 0$. We use $\beta_{ijkl} \in \{0, 1\}$ to represent the task offloading decision. If $\beta_{ijkl} = 1$, task $ij$ is decided to be offloaded from vehicle $ij$ to RSU $j$, and then forwarded to RSU $k$ and computed on it. Finally, the task result is transmitted from RSU $k$ to RSU $l$, and then to vehicle $ij$ that is currently covered by RSU $l$. Otherwise, $\beta_{ijkl} = 0$. Note that: 1) when $j = k$, $\beta_{ijkl} = 1$ means that task $ij$ is computed on RSU $j$ without using R2R task offloading, and then the task result is transmitted to RSU $l$; 2) when $j = l$, $\beta_{ijkl} = 1$ means that task result $ij$ is transmitted from RSU $k$ to the original RSU $j$, and finally returned to the vehicle from the RSU. It can be seen that our scheme supports the assumptions of the works in [4], [5], [6], [7], and [8] via the above two cases.

Considering the task profiles and the driving speeds of vehicles in urban streets, when the task processing is completed, vehicle $ij$ can only reach the next RSU at the farthest in the driving path of vehicle $ij$ starting from its original RSU $j$. We use $\xi_{ij}$ to represent the index of the next RSU and use $\chi_{ij}$ to represent the holding time of vehicle $ij$ with regard to RSU $j$. $\chi_{ij}$ is the duration that vehicle $ij$ can maintain the communication with its original RSU $j$ before it moves out of the coverage of RSU $j$. Thus, we can define the value range of index $l$ in $\beta_{ijkl}$ as $l \in \mathcal{L}_{ij} = \{j, \xi_{ij}\}$.

We use two sets $\mathcal{A} = \{\alpha_{ij}\}_{i \in \mathcal{N}_j, j \in \mathcal{M}}$ and $\mathcal{B} = \{\beta_{ijkl}\}_{i \in \mathcal{N}_j, j,k \in \mathcal{M}, l \in \mathcal{L}_{ij}}$ to include the task processing decision indicators of all the vehicles. We have:

$$C_1 : \alpha_{ij} \in \{0, 1\}, i \in \mathcal{N}_j, j \in \mathcal{M}, \tag{1}$$

$$C_2 : \beta_{ijkl} \in \{0, 1\}, i \in \mathcal{N}_j, j, k \in \mathcal{M}, l \in \mathcal{L}_{ij}, \tag{2}$$

$$C_3 : \alpha_{ij} + \sum_{k \in \mathcal{M}} \sum_{l \in \mathcal{L}_{ij}} \beta_{ijkl} = 1, i \in \mathcal{N}_j, j \in \mathcal{M}, \tag{3}$$

where $C_1$ and $C_2$ indicate the value ranges of the task processing decision indicators, and $C_3$ describes the task of a vehicle can be arranged to only one task processing option, namely, a vehicle can choose only one decision from $\mathcal{A}$ and $\mathcal{B}$.

### B. Computing Model

*1) Computing on a Vehicle:* If $\alpha_{ij} = 1$, vehicle $ij$ is decided to compute its task locally. We denote $F_{ij}^{v}$ as the total computing resource of vehicle $ij$. Let $f_{ij}^{v}$ represent the computing resource allocated from $F_{ij}^{v}$ to compute task $ij$. $f_{ij}^{v}$ must not exceed $F_{ij}^{v}$, so we have

$$C_4 : f_{ij}^{v} \in [0, F_{ij}^{v}], i \in \mathcal{N}_j, j \in \mathcal{M}. \tag{4}$$

Thus, we can formulate the local computing delay of task $ij$ as

$$t_{ij}^{v} = \frac{c_{ij}}{f_{ij}^{v}}. \tag{5}$$

We include the local computing resource allocation indicators of all the vehicles into a set $\mathcal{F}^{v} = \{f_{ij}^{v}\}_{i \in \mathcal{N}_j, j \in \mathcal{M}}$.

*2) Computing on an RSU:* If $\beta_{ijkl} = 1$, vehicle $ij$ is decided to offload its task to RSU $k$ and compute on it. We denote $F_{k}^{r}$ as the total computing resource of RSU $k$. Let $f_{ijk}^{r}$ represent the computing resource allocated from $F_{k}^{r}$ to compute task $ij$. $f_{ijk}^{r}$ must not exceed $F_{k}^{r}$, so we have

$$C_5 : f_{ijk}^{r} \in [0, F_{k}^{r}], i \in \mathcal{N}_j, j, k \in \mathcal{M}. \tag{6}$$

Multiple tasks may be offloaded to RSU $j$ and computed on it. Thus, the sum of the allocated computing resource must

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

equal to $F_k^r$, and then we have

$$C_6: \sum_{i\in\mathcal{N}_j, j\in\mathcal{M}} f_{ijk}^r \le F_k^r, k \in \mathcal{M}. \tag{7}$$

We can formulate the computing delay of task $ij$ on RSU $k$ as

$$t_{ijk}^r = \frac{c_{ij}}{f_{ijk}^r}. \tag{8}$$

We include the computing resource allocation indicators of all the RSUs into a set $\mathcal{F}^r = \{f_{ijk}^r\}_{i\in\mathcal{N}_j, j,k\in\mathcal{M}}$.

### C. Communication Model

We assume the radio access system of each RSU is based on orthogonal frequency division multiple access (OFDMA). OFDMA (Orthogonal Frequency Division Multiple Access) is a communication technique that divides the available spectrum into multiple subcarriers and assigns them to the vehicles, enabling parallel transmission.

*1) Vehicle to RSU (V2R) Transmission:* We can formulate the SINR (signal to interference plus noise ratio) of the channel for the V2R transmission from vehicle $ij$ to RSU $j$. It is given by

$$\Gamma_{ij}^{v2r} = \frac{p_{ij}^{v2r} H_{ij}^{v2r}}{N_0 W + I_{ij}^{v2r}}, \tag{9}$$

where $H_{ij}^{v2r}$ is the channel gain and $p_{ij}^{v2r}$ is the transmit power of vehicle $ij$ used for V2R transmission. The channel bandwidth is $W$ and the additive Gaussian white noise power spectral density is defined as $N_0$, so $N_0 W$ is the additive Gaussian white noise power of the channel. Owing to the channel orthogonality of OFDMA, there are no inter-channel inference among the channels of the RSU, however, co-channel interference exists, which is generated on the same channels of all RSUs. We denote the co-channel interference for the vehicle $ij$-to-RSU $j$ transmission as $I_{ij}^{v2r}$. The maximum transmit power of vehicle $ij$ is denoted as $P_{ij}^{v2r}$. $p_{ij}^{v2r}$ must not exceed $P_{ij}^{v2r}$, so we have

$$C_7: p_{ij}^{v2r} \in [0, P_{ij}^{v2r}], i \in \mathcal{N}_j, j \in \mathcal{M}. \tag{10}$$

Based on the Shannon formula, we can calculate the V2R transmission rate for the uplink task transmission from vehicle $ij$ to RSU $j$ as

$$r_{ij}^{v2r} = W \log(1 + \Gamma_{ij}^{v2r}). \tag{11}$$

Thus, the V2R task transmission delay of vehicle $ij$ can be given by

$$t_{ij}^{v2r} = \frac{d_{ij}^{in}}{r_{ij}^{v2r}}. \tag{12}$$

We collect the transmit power allocation indicators of all the vehicles into a set $\mathcal{P} = \{p_{ij}^{v2r}\}_{i\in\mathcal{N}_j, j\in\mathcal{M}}$.

*2) RSU to Vehicle (R2V) Transmission:* We can formulate the SINR of the channel for the R2V transmission from RSU $l$ ($l \in \mathcal{L}_{ij}$) to vehicle $ij$. It is given by

$$\Gamma_{ijl}^{r2v} = \frac{P_{ijl}^{r2v} H_{ijl}^{r2v}}{N_0 W + I_{ijl}^{r2v}}, \tag{13}$$

where $P_{ijl}^{r2v}$ is the transmit power of RSU $l$ for transmitting the task result to vehicle $ij$, $H_{ijl}^{r2v}$ is the channel gain, $N_0 W$ is the additive Gaussian white noise power of the channel, and $I_{ijl}^{r2v}$ is the co-channel interference. There is no inter-channel interference due to the channel orthogonality of OFDMA. Thus, we can formulate the R2V transmission rate for the downlink transmission from RSU $l$ to vehicle $ij$ as

$$r_{ijl}^{r2v} = W \log(1 + \Gamma_{ijl}^{r2v}). \tag{14}$$

Then, we have the R2V task result transmission delay from RSU $l$ to vehicle $ij$

$$t_{ijl}^{r2v} = \frac{d_{ij}^{out}}{r_{ijl}^{r2v}}. \tag{15}$$

*3) RSU-to-RSU (R2R) Transmission:* We denote the total wired transmission rate from RSU $j$ to the other RSUs as $R_j^{r2r}$. $r_{jk}^{r2r}$ is the transmission rate allocated from $R_j^{r2r}$ for the connection from RSU $j$ to RSU $k$. $r_{jk}^{r2r}$ must not exceed $R_j^{r2r}$, so we have

$$C_8: r_{jk}^{r2r} \in [0, R_j^{r2r}], j, k \in \mathcal{M}, k \ne j. \tag{16}$$

Thus, the R2R transmission delay for transmitting task $ij$ from RSU $j$ to RSU $k$ is

$$t_{ijk}^{in} = \frac{d_{ij}^{in}}{r_{jk}^{r2r}}. \tag{17}$$

Correspondingly, the R2R transmission delay for transmitting task result $ij$ from RSU $k$ to RSU $l$ is

$$t_{ijkl}^{out} = \frac{d_{ij}^{out}}{r_{kl}^{r2r}}. \tag{18}$$

We include all the transmission rate allocation indicators into a set $\mathcal{R} = \{r_{jk}^{r2r}\}_{j,k\in\mathcal{M},k\ne j}$.

### D. Energy Consumption Model

*1) Energy Consumption of Local Task Processing:* For vehicle $ij$, the energy consumption of local task processing is the energy consumed in the task computing process on the vehicle. It is given by

$$e_{ij}^v = \kappa_{ij} (f_{ij}^v)^2 c_{ij}, \tag{19}$$

where $\kappa_{ij}$ is a constant indicating the effective switched capacitance, which depends on the chip architecture [31].

*2) Energy Consumption of Task Offloading:* For vehicle $ij$, the energy consumption of task offloading only includes energy consumed in the V2R task transmission process. It is expressed by

$$e_{ij}^{v2r} = p_{ij}^{v2r} t_{ij}^{v2r}. \tag{20}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

### E. Task Processing Costs

We define the task processing cost of the task of a vehicle as a weighted sum of the task processing delay and energy consumption. $w_{ij}$ is the weight factor for vehicle $ij$.

Based on the analysis in the above subsections, there are 5 task processing cases, shown in Fig. 1:

*1) Task Is Processed Locally:* In this case, $\alpha_{ij} = 1$, the task processing delay of vehicle $ij$ is $t_{ij}^{v}$ and the energy consumption is $e_{ij}^{v}$. Thus, we have the task processing cost in this case

$$u_{ij}^{(1)} = \alpha_{ij}(t_{ij}^{v} + w_{ij}e_{ij}^{v}). \tag{21}$$

*2) Task Is Processed on Original RSU and Vehicle Has Not Left:* In this case, $\beta_{ijjj} = 1$, namely, the task processing delay of vehicle $ij$ consists of the V2R task transmission delay from vehicle $ij$ to RSU $j$, $t_{ij}^{v2r}$, the task computing delay on RSU $j$, $t_{ijj}^{r}$, and the R2V task result transmission delay from RSU $j$ to vehicle $ij$, $t_{ijj}^{r2v}$; the energy consumption is the energy consumed in the V2R transmission process, $e_{ij}^{v2r}$. Moreover, vehicle $ij$ must be under the coverage of RSU $j$ before it receives the task result sent from the RSU. This constraint can be formulated as

$$C_9 : \beta_{ijjj}(t_{ij}^{v2r} + t_{ijj}^{r} + t_{ijj}^{r2v}) \le \chi_{ij}, i \in \mathcal{N}_j, j \in \mathcal{M}. \tag{22}$$

Thus, we have the task processing cost in this case

$$u_{ij}^{(2)} = \beta_{ijjj}(t_{ij}^{v2r} + t_{ijj}^{r} + t_{ijj}^{r2v} + w_{ij}e_{ij}^{v2r}). \tag{23}$$

*3) Task Is Processed on Original RSU and Vehicle Has Left:* In this case, $\beta_{ijj\xi_{ij}} = 1$, that is, the task processing delay of vehicle $ij$ consists of the V2R task transmission delay from vehicle $ij$ to RSU $j$, $t_{ij}^{v2r}$, the task computing delay on RSU $j$, $t_{ijj}^{r}$, the R2R task result transmission delay from RSU $j$ to RSU $\xi_{ij}$, $t_{ijj\xi_{ij}}^{out}$, and the R2V task result transmission delay from RSU $\xi_{ij}$ to vehicle $ij$, $t_{ij\xi_{ij}}^{r2v}$; the energy consumption is the energy consumed in the V2R transmission process, $e_{ij}^{v2r}$. Additionally, vehicle $ij$ has moved to the next RSU $\xi_{ij}$, and then it receives the task result sent from the RSU. This constraint can be formulated as

$$C_{10} : \beta_{ijj\xi_{ij}}\chi_{ij} < t_{ij}^{v2r} + t_{ijj}^{r} + t_{ijj\xi_{ij}}^{out} + t_{ij\xi_{ij}}^{r2v}, i \in \mathcal{N}_j, j \in \mathcal{M}. \tag{24}$$

Thus, we have the task processing cost in this case

$$u_{ij}^{(3)} = \beta_{ijj\xi_{ij}}(t_{ij}^{v2r} + t_{ijj}^{r} + t_{ijj\xi_{ij}}^{out} + t_{ij\xi_{ij}}^{r2v} + w_{ij}e_{ij}^{v2r}). \tag{25}$$

*4) Task Is Processed on Another RSU and Vehicle Has Not Left Original RSU:* In this case, $\beta_{ijkj} = 1, k \ne j$, namely, the task processing delay of vehicle $ij$ consists of the V2R task transmission delay from vehicle $ij$ to RSU $j$, $t_{ij}^{v2r}$, the R2R task transmission delay from RSU $j$ to RSU $k$, $t_{ijk}^{in}$, the task computing delay on RSU $k$, $t_{ijk}^{r}$, the R2R task result transmission delay from RSU $k$ to RSU $j$, $t_{ijkj}^{out}$, and the R2V task result transmission delay from RSU $j$ to vehicle $ij$, $t_{ijj}^{r2v}$; the energy consumption is the energy consumed in the V2R transmission process, $e_{ij}^{v2r}$. Moreover, vehicle $ij$ must be under the coverage of RSU $j$ before it receives the task result sent from the RSU. This constraint can be formulated as

$$C_{11} : \beta_{ijkj}(t_{ij}^{v2r} + t_{ijk}^{in} + t_{ijk}^{r} + t_{ijkj}^{out} + t_{ijj}^{r2v}) \le \chi_{ij}, \quad i \in \mathcal{N}_j, j, k \in \mathcal{M}, k \ne j. \tag{26}$$

Thus, we have the task processing cost in this case

$$u_{ijk}^{(4)} = \beta_{ijkj}(t_{ij}^{v2r} + t_{ijk}^{in} + t_{ijk}^{r} + t_{ijkj}^{out} + t_{ijj}^{r2v} + w_{ij}e_{ij}^{v2r}). \tag{27}$$

*5) Task Is Processed on Another RSU and Vehicle Has Left Original RSU:* In this case, $\beta_{ijk\xi_{ij}} = 1, k \ne j$, that is, the task processing delay of vehicle $ij$ consists of the V2R task transmission delay from vehicle $ij$ to RSU $j$, $t_{ij}^{v2r}$, the R2R task transmission delay from RSU $j$ to RSU $k$, $t_{ijk}^{in}$, the task computing delay on RSU $k$, $t_{ijk}^{r}$, the R2R task result transmission delay from RSU $k$ to RSU $\xi_{ij}$, $t_{ijk\xi_{ij}}^{out}$, and the R2V task result transmission delay from RSU $\xi_{ij}$ to vehicle $ij$, $t_{ij\xi_{ij}}^{r2v}$; the energy consumption is the energy consumed in the V2R transmission process, $e_{ij}^{v2r}$. Additionally, vehicle $ij$ has moved to the next RSU $\xi_{ij}$, and then it receives the task result sent from the RSU. This constraint can be formulated as

$$C_{12} : \beta_{ijk\xi_{ij}}\chi_{ij} < t_{ij}^{v2r} + t_{ijk}^{in} + t_{ijk}^{r} + t_{ijk\xi_{ij}}^{out} + t_{ij\xi_{ij}}^{r2v}, \quad i \in \mathcal{N}_j, j, k \in \mathcal{M}, k \ne j. \tag{28}$$

Thus, we have the task processing cost in this case

$$u_{ijk}^{(5)} = \beta_{ijk\xi_{ij}}(t_{ij}^{v2r} + t_{ijk}^{in} + t_{ijk}^{r} + t_{ijk\xi_{ij}}^{out} + t_{ij\xi_{ij}}^{r2v} + w_{ij}e_{ij}^{v2r}). \tag{29}$$

Finally, we summarize the above 5 cases and give the total task processing cost of all the vehicles as

$$u = \sum_{j\in\mathcal{M}}\sum_{i\in\mathcal{N}_j}\Big(u_{ij}^{(1)} + u_{ij}^{(2)} + u_{ij}^{(3)} + \sum_{k\in\mathcal{M},k\ne j}\big(u_{ijk}^{(4)} + u_{ijk}^{(5)}\big)\Big). \tag{30}$$

### F. Problem Formulation

The objective of our scheme is to minimize the total task processing cost of all the vehicles by jointly optimizing the task processing decisions in $\mathcal{A}$ and $\mathcal{B}$, the computing resource allocation indicators in $\mathcal{F}^v$ and $\mathcal{F}^r$, the V2R transmit power indicators in $\mathcal{P}$, and the R2R transmission rate indicators in $\mathcal{R}$. We construct our optimization problem as follows

$$P_1 : \underset{\mathcal{A},\mathcal{B},\mathcal{F}^v,\mathcal{F}^r,\mathcal{P},\mathcal{R}}{\text{minimize}} \; u \quad s.t. \quad C_1 - C_{12}. \tag{31}$$

## IV. TD3-Based DRL Algorithm

The proposed problem is a mixed-integer nonlinear programming problem because: 1) the variables are continuous-discrete-mixed; 2) the objective function is nonconvex; 3) constraints $C_9 - C_{12}$ are nonconvex. Therefore, it is very difficult to solve the problem directly by using numerical methods. To this end, we design a TD3-based DRL algorithm to provide a near-optimal solution while guaranteeing the online performance of the algorithm. To reduce the complexity

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

of the training process of our algorithm, we embed an optimization subroutine to solve the computing resource allocation sub-problem and the transmit power allocation sub-problem for the corresponding vehicles by using numerical methods, then let the TD3 part of our algorithm handle the rest of the optimization problem, thus decreasing the searching space of the TD3 model.

### A. Optimization Subroutine via Numerical Methods

From the structure of the original problem $P_1$, we can find that, if the task processing decision of vehicle $ij$ is made, the computing resource $f_{ij}^v$ and the transmit power $p_{ij}^{v2r}$ of the vehicle can be naturally decoupled with the other variables.

*1) Computing Resource Allocation Sub-Problem of Vehicle $ij$:* If $\alpha_{ij} = 1$, task $ij$ is processed locally. In this case, based on Constraint $C_3$ (see Equation (3)), the values of the other task processing options of vehicle $ij$ are all 0, so $u_{ij}^{(2)} + u_{ij}^{(3)} + \sum_{k\in\mathcal{M},k\neq j}\left(u_{ijk}^{(4)} + u_{ijk}^{(5)}\right) = 0$ in the objective function, and then $u_{ij}^{(1)}$ and Constraint $C_{13}$ can be decoupled from $P_1$. Thus, the computing resource allocation sub-problem of vehicle $ij$ can be decomposed and formulated as

$$\begin{aligned} SP_1: \underset{f_{ij}^v}{\text{minimize}}\ & u_{ij}^{(1)} \\ & = t_{ij}^v + w_{ij}e_{ij}^v = \frac{c_{ij}}{f_{ij}^v} + w_{ij}\kappa_{ij}(f_{ij}^v)^2 c_{ij} \\ s.t.\quad & C_{13}: f_{ij}^v \in [0, F_{ij}^v] \end{aligned} \tag{32}$$

This sub-problem is convex since 1) the objective function is a positive sum of a $\frac{1}{x}$-like convex structure and a $x^2$-like convex structure; 2) the constraint $C_{13}$ is linear.

Here, we design a fast numerical method to solve $SP_1$ optimally. Firstly, we obtain the first-order derivative of the objective function, expressed as $\psi_{ij} = -\frac{c_{ij}}{(f_{ij}^v)^2} + 2w_{ij}\kappa_{ij}f_{ij}^v c_{ij}$. Let $\psi_{ij} = 0$ and then we get the optimal solution to the equation as $\bar{p}_{ij}^v = \frac{1}{\sqrt[3]{2w_{ij}\kappa_{ij}}}$. Finally, we check whether $\bar{p}_{ij}^v$ satisfies the constraint $C_{13}$. If $\bar{p}_{ij}^v > F_{ij}^v$, then the optimal solution to $SP_1$ is $p_{ij}^{v*} = F_{ij}^v$, else $p_{ij}^{v*} = \bar{p}_{ij}^v$. The details of the method are shown in **Algorithm 1**.

**Algorithm 1** Fast Numerical Method for $SP_1$

```
Obtain p̄_ij^v = 1 / ∛(2 w_ij κ_ij);
if p̄_ij^v > F_ij^v then
    p_ij^{v*} = F_ij^v;
else
    p_ij^{v*} = p̄_ij^v;
end if
```

*2) Transmit Power Allocation Sub-Problem of Vehicle $ij$:* If $\alpha_{ij} = 0$, task $ij$ is offloaded to an RSU and vehicle $ij$ needs to transmit the task to its affiliated RSU. In this case, only one of the other task processing options is valued as 1, so the V2R transmission cost (represented by $t_{ij}^{v2r} + w_{ij}e_{ij}^{v2r}$ in $u_{ij}^{(2)}$, $u_{ij}^{(3)}$, $u_{ijk}^{(4)}$, or $u_{ijk}^{(5)}$) can be decoupled from $P1$. Thus, we can decompose the transmit power allocation sub-problem of vehicle $ij$. It is formulated as

$$\begin{aligned} SP_2: \underset{p_{ij}^v}{\text{minimize}}\ & t_{ij}^{v2r} + w_{ij}e_{ij}^{v2r} \\ & = \frac{(1 + w_{ij}p_{ij}^{v2r})d_{ij}^{in}}{W\log(1 + \frac{p_{ij}^{v2r}H_{ij}^{v2r}}{N_0 W + I_{ij}^{v2r}})} \\ s.t.\quad & C_{14}: p_{ij}^{v2r} \in [0, P_{ij}^{v2r}] \end{aligned} \tag{33}$$

It can be seen that $SP_2$ is a quasi-convex optimization problem because the objective function consists of a linear numerator and a concave denominator concerning $p_{ij}^{v2r}$, and $C_{14}$ is linear. Therefore, it can be optimally solved by existing fractional programming methods, such as the Dinkelbach algorithm [32].

### B. TD3-Based DRL Algorithm Framework

The DRL framework is very suitable for solving real-time complex decision-making problems, and its purpose is to obtain the best correspondence between actions and states. Removing $SP_1$ and $SP_2$ from $P_1$, we formulate the rest of $P_1$ as a Markov decision process model, which mainly includes three key elements: state space, action space, and reward function. We also add a penalty function into the reward function to satisfy the constraints of the optimization problem.

*1) State:* A state $\boldsymbol{s}$ consists of the information on task profiles, vehicle movements, and wireless conditions. We denote it as

$$\boldsymbol{s} = \{\mathcal{V}, \mathcal{G}, \mathcal{H}\}, \tag{34}$$

where the set $\mathcal{V} = \{< c_{ij}, d_{ij}^{in}, d_{ij}^{out} >\}_{i\in\mathcal{N}_j, j\in\mathcal{M}}$ includes the task profiles of all the vehicles, $\mathcal{G} = \{< \xi_{ij}, \chi_{ij} >\}_{i\in\mathcal{N}_j, j\in\mathcal{M}}$ includes the indices of the next RSUs and the holding time of all the vehicles, and $\mathcal{H} = \{< H_{ij}^{v2r}, H_{ijl}^{r2v}, I_{ij}^{v2r}, I_{ijl}^{r2v}, N_0 >\}_{i\in\mathcal{N}_j, j,l\in\mathcal{M}}$ includes the wireless environment parameters of all the RSUs.

*2) Action:* An action $\boldsymbol{a}$ contains the task processing decisions for all the vehicles, the computing resource allocation decisions of all the RSUs, and the R2R transmission rate allocation decisions of all the RSUs. It is expressed as

$$\boldsymbol{a} = \{\mathcal{A}, \mathcal{B}, \mathcal{F}^r, \mathcal{R}\}. \tag{35}$$

Note that, $\boldsymbol{a}$ does not include $\mathcal{F}^v$ and $\mathcal{P}$ because they are solved by the optimization subroutine.

*3) Reward:* The reward of a standard DRL algorithm is to maximize an objective function. Thus, we set the reward $r = -u$, which is the opposite of $u$.

*4) Penalty:* An action needs to satisfy the constraints of the optimization problem. Constraints $C_1$, $C_2$, $C_4$, $C_5$, $C_7$ and $C_8$ can be directly handled by using the $tanh$ function as the activation of the output of the DRL model; constraints $C_3$ and $C_6$ can also be directly handled by using the $softmax$ function. When an action violates constraints $C_9 - C_{12}$, we take the task processing delay as the penalty value to reduce the reward gained by the DRL model. The penalties of task processing case 2)-5) (See Section III-E) corresponding to $C_9 - C_{12}$ are given as follows:

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

a) *Violation of Case 2)*: If $\beta_{ijjj}=1$ and the task processing delay $t_{ij}^{(2)}=\beta_{ijjj}(t_{ij}^{v2r}+t_{ijj}^{r}+t_{ijj}^{r2v})>\chi_{ij}$, the action for vehicle $ij$ violates constraint $C_9$. That is, vehicle $ij$ is decided to process its task as Case 2), however, its task processing is not completed before the vehicle moves out of the coverage of RSU $j$. The penalty for this condition is

$$\rho_{ij}^{(2)}=\begin{cases}0, & t_{ij}^{(2)}\le\chi_{ij},\\ t_{ij}^{(2)}, & t_{ij}^{(2)}>\chi_{ij}.\end{cases} \tag{36}$$

b) *Violation of Case 3)*: If $\beta_{ijj\xi_{ij}}=1$ and the task processing delay $t_{ij}^{(3)}=t_{ij}^{v2r}+t_{ijj}^{r}+t_{ijj\xi_{ij}}^{out}+t_{ij\xi_{ij}}^{r2v}\le\chi_{ij}$, the action for vehicle $ij$ violates constraint $C_{10}$. That is, vehicle $ij$ is decided to process its task as Case 3), however, its task processing is completed before the vehicle moves into the coverage of RSU $\xi_{ij}$. The penalty for this condition is

$$\rho_{ij}^{(3)}=\begin{cases}0, & t_{ij}^{(3)}>\chi_{ij},\\ t_{ij}^{(3)}, & t_{ij}^{(3)}\le\chi_{ij}.\end{cases} \tag{37}$$

c) *Violation of Case 4)*: If $\beta_{ijkj}=1$ and the task processing delay $t_{ijk}^{(4)}=t_{ij}^{v2r}+t_{ijk}^{in}+t_{ijk}^{r}+t_{ijkj}^{out}+t_{ijj}^{r2v}>\chi_{ij}$, the action for vehicle $ij$ violates constraint $C_{11}$. That is, vehicle $ij$ is decided to process its task as Case 4), however, its task processing is not completed before the vehicle moves out of the coverage of RSU $j$. The penalty for this condition is

$$\rho_{ijk}^{(4)}=\begin{cases}0, & t_{ijk}^{(4)}\le\chi_{ij},\\ t_{ijk}^{(4)}, & t_{ijk}^{(4)}>\chi_{ij}.\end{cases} \tag{38}$$

d) *Violation of Case 5)*: If $\beta_{ijk\xi_{ij}}=1$ and the task processing delay $t_{ijk}^{(5)}=t_{ij}^{v2r}+t_{ijk}^{in}+t_{ijk}^{r}+t_{ijk\xi_{ij}}^{out}+t_{ij\xi_{ij}}^{r2v}\le\chi_{ij}$, the action for vehicle $ij$ violates constraint $C_{12}$. That is, vehicle $ij$ is decided to process its task as Case 5), however, its task processing is completed before the vehicle moves into the coverage of RSU $\xi_{ij}$. The penalty for this condition is

$$\rho_{ijk}^{(5)}=\begin{cases}0, & t_{ijk}^{(5)}>\chi_{ij},\\ t_{ijk}^{(5)}, & t_{ijk}^{(5)}\le\chi_{ij}.\end{cases} \tag{39}$$

We integrate the penalties of the 4 task processing cases into the reward function. Then, the reward function can be rewritten as

$$r=-u-\sum_{j\in\mathcal{M}}\sum_{i\in\mathcal{N}_j}\left(\rho_{ij}^{(2)}+\rho_{ij}^{(3)}+\sum_{k\in\mathcal{M},k\neq j}\left(\rho_{ijk}^{(4)}+\rho_{ijk}^{(5)}\right)\right). \tag{40}$$

Our TD3-based DRL algorithm is extended from the standard TD3 algorithm [33]. TD3 is based on the Deep Deterministic Policy Gradient (DDPG) algorithm, which leverages an actor-critic architecture to solve our optimization problem efficiently. This architecture is also suitable for optimizing discrete variables and continuous ones. An actor neural network is used to output an action based on the environment input. The action represents the solution to the optimization problem. A critic neural network is in charge of evaluating the action and it helps the actor to improve the quality of its solution to the problem. The actors and critics are trained by the samples collected through interacting with the environment and finally, they converge after multiple episodes.

TD3 has three advanced techniques compared with DDPG: 1) Double Networks. TD3 employs two sets of critic networks and takes the minimum value between them as the target value to suppress the issue of network overestimation caused by bootstrapping and maximization. 2) Target Policy Smoothing Regularization. To reduce the large variances and inaccuracy in target estimation when updating the critic networks, TD3 introduces a regularization strategy called target policy smoothing, which adds a small amount of random noise to the target action. 3) Delayed Update. TD3 updates the actor network after updating the critic network multiple times, avoiding a highly unstable critic network that leads to oscillations in the actor network.

TD3 uses a primary actor network $\pi^P(\boldsymbol{s}|\theta_\pi^P)$, a target actor network $\pi^T(\boldsymbol{s}|\theta_\pi^T)$, two primary critic networks $Q_1^P(\boldsymbol{s},\boldsymbol{a},\mathcal{F}^{v*},\mathcal{P}^*|\theta_{Q_1}^P)$ and $Q_2^P(\boldsymbol{s},\boldsymbol{a},\mathcal{F}^{v*},\mathcal{P}^*|\theta_{Q_2}^P)$, and two target critic networks $Q_1^T(\boldsymbol{s},\boldsymbol{a},\mathcal{F}^{v*},\mathcal{P}^*|\theta_{Q_1}^T)$ and $Q_2^T(\boldsymbol{s},\boldsymbol{a},\mathcal{F}^{v*},\mathcal{P}^*|\theta_{Q_2}^T)$ to construct its actor-critic architecture. Note that, $\mathcal{F}^{v*}$ and $\mathcal{P}^*$ contain the optimal computing resource allocation and the transmit power allocation of all the vehicles, which are solved by the optimization subroutine. The actor network determines and obtains the action to be performed based on the current state; the critic network evaluates the Q-value based on the current state, action, and $\mathcal{F}^{v*}$ and $\mathcal{P}^*$. Besides, TD3 uses a replay buffer $\mathcal{Z}$ to store the history transitions.

According to $\boldsymbol{s}$, $\pi^P$ makes an action $\boldsymbol{a}$ with an additive exploration noise $\delta^P$, which is sampled from a clipped normal distribution. $\boldsymbol{a}$ is given by

$$\boldsymbol{a}=\pi^P(\boldsymbol{s}|\theta_\pi^P)+\delta^P. \tag{41}$$

Then, based on $\boldsymbol{a}$, we get $\mathcal{F}^{v*}$ and $\mathcal{P}^*$ by solving the computing resource allocation sub-problem (32) and the transmit power allocation sub-problem (33) of each vehicle, respectively. Next, $\boldsymbol{a}$, $\mathcal{F}^{v*}$, and $\mathcal{P}^*$ are input to the environment to get the reward $r$, and then the state moves to the next state $\boldsymbol{s}'$. The current transition tuple $\{\boldsymbol{s},\boldsymbol{a},\mathcal{F}^{v*},\mathcal{P}^*,r,\boldsymbol{s}'\}$ is then stored in the experience replay buffer $\mathcal{Z}$.

$\pi^P$ is updated by applying the sampled policy gradient based on the value estimations of $Q_1^P$. $Q_1^P$ and $Q_2^P$ are updated by minimizing their temporal-difference (TD) error loss functions [33], shown as follows:

$$L(\theta_{Q_k}^P)=\mathbb{E}\left(\left(v-Q_k^P(\boldsymbol{s},\boldsymbol{a},\mathcal{F}^{v*},\mathcal{P}^*|\theta_{Q_k}^P)\right)^2\right),\quad k=1,2, \tag{42}$$

where the target Q-value $v$ is obtained by selecting the smaller one from the estimations of $Q_1^T$ and $Q_2^T$. $v$ is also added by a noise sample $\delta^T$ from a clipped normal distribution. $v$ is given by

$$v=r+\eta\min_{k=1,2}Q_k^T\left(\boldsymbol{s}',\pi^T\left(\boldsymbol{s}'\right)+\delta^T,\mathcal{F}^{v*'},\mathcal{P}^{*'}|\theta_{Q_k}^T\right), \tag{43}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

where $\eta$ is a discounting factor, $\mathcal{F}^{v*'}$ and $\mathcal{P}^{*'}$ are the computing resource allocation and the transmit power allocation of all the vehicles corresponding to the action $\pi^T\left(\boldsymbol{s}'\right)+\delta^T$.

The training process begins when $\mathcal{Z}$ is full. In each training step, the parameters $\theta_\pi^P$ of $\pi^P$, $\theta_{Q_1}^P$ of $Q_1^P$, and $\theta_{Q_2}^P$ of $Q_2^P$ are updated by a mini-batch of $E$ transitions from $\mathcal{Z}$. These updates follow the sampled policy gradient method [33], described as

$$\theta_\pi^P=\theta_\pi^P-\frac{\phi^\pi}{E}\sum_{\epsilon=1}^{E}\Big(\nabla_{\boldsymbol{a}}Q_1^P\left(\boldsymbol{s}(\epsilon),\pi(\boldsymbol{s}(\epsilon)|\theta_{Q_1}^P)\right)\nabla_{\theta_\pi^P}\pi\left(\boldsymbol{s}(\epsilon)|\theta_\pi^P\right)\Big), \tag{44}$$

$$\theta_{Q_k}^P=\theta_{Q_k}^P-\frac{\phi^Q}{E}\sum_{\epsilon=1}^{E}\Big(2\Big(v(\epsilon)-Q_k^P\left(\boldsymbol{s}(\epsilon),\boldsymbol{a}(\epsilon),\mathcal{F}^{v*}(\epsilon),\mathcal{P}^*(\epsilon)|\theta_{Q_k}^P\right)\Big)\nabla_{\theta_{Q_1}^P}Q_k^P\left(\boldsymbol{s}(\epsilon),\boldsymbol{a}(\epsilon),\mathcal{F}^{v*}(\epsilon),\mathcal{P}^*(\epsilon)|\theta_{Q_k}^P\right)\Big),\quad k=1,2, \tag{45}$$

where $\phi^\pi$ is the learning rate of $\pi^P$, and $\phi^Q$ is the learning rate of $Q_1^P$ and $Q_2^P$.

The parameters $\theta_\pi^T$ of $\pi^T$, $\theta_{Q_1}^T$ of $Q_1^T$, and $\theta_{Q_2}^T$ of $Q_2^T$ are updated via a soft update mechanism [33], which is a linear weighted sum with the parameters of their corresponding primary networks. These updates follow:

$$\theta_\pi^T=\lambda\theta_\pi^T+(1-\lambda)\theta_\pi^P, \tag{46}$$

$$\theta_{Q_k}^T=\lambda\theta_{Q_k}^T+(1-\lambda)\theta_{Q_k}^P,\quad k=1,2, \tag{47}$$

where $\lambda\in(0,1)$ is the soft update weight. It can be seen that the new parameters of a target network are a weighted sum of its original parameters and the parameters of its corresponding primary network.

The detailed process of the TD3-based DRL algorithm is described in **Algorithm 2**.

**Algorithm 2** TD3-Based DRL Algorithm

```
Randomly initialize the parameters of the network π^P, Q_1^P, and Q_2^P;
Initialize the network π^T, Q_1^T, and Q_2^T by θ_π^T ← θ_π^P, θ_{Q_1}^T ← θ_{Q_1}^P, and θ_{Q_2}^T ← θ_{Q_2}^P;
Initialize the reply buffer Z;
for each episode do
    Reset the environment;
    for each episode do
        Observe the current state s(τ);
        Generate an action a(τ) with exploration noise;
        Obtain F^{v*'} by solving the computing resource allocation sub-problem (32);
        Obtain P^{*'} by solving the transmit power allocation sub-problem (33);
        Obtain the reward r(τ);
        Move the environment to the next state s'(τ);
        Store the transition tuple {s, a, F^{v*}, P^*, r, s'} into the replay buffer Z;
        if Z is full then
            Sample a mini-batch of E transitions from Z;
            Update Q_1^P and Q_2^P by minimizing their TD errors;
            if τ mod ϒ then
                \\ update every ϒ steps
                Update π^P via the policy gradient;
                Update π^T, Q_1^T and Q_2^T via the soft update mechanism;
            end if
        end if
    end for
end for
```

### C. Algorithm Execution

When the training process of the algorithm is completed, the primary actor network is trained and then it can be used to output the solution $\mathcal{A}^*$, $\mathcal{B}^*$, $\mathcal{F}^{r*}$, and $\mathcal{R}^*$. Then, substituting the above solution to sub-problem $SP_1$ and $SP_2$ for each vehicle, we can obtain $\mathcal{F}^{v*}$ and $\mathcal{P}^*$ via the proposed numerical methods (i.e., **Algorithm 1** and the Dinkelbach algorithm), respectively.

### D. Algorithm Deployment

Our algorithm is deployed on the network controller which is equipped at the edge layer of the network and is connected with all the RSUs. The controller continuously collects the state information $\mathcal{V}$, $\mathcal{G}$, and $\mathcal{H}$, which are sent from the vehicles and are sensed from the wireless environment of each RSU, respectively. The algorithm executes and outputs the values in $\mathcal{A}^*$, $\mathcal{B}^*$, $\mathcal{F}^{r*}$, $\mathcal{R}^*$, $\mathcal{F}^{v*}$, and $\mathcal{P}^*$. Then, $\mathcal{A}^*$, $\mathcal{B}^*$, $\mathcal{F}^{v*}$, and $\mathcal{P}^*$ are sent to the corresponding vehicles, and $\mathcal{F}^{r*}$ and $\mathcal{R}^*$ are sent to the corresponding RSUs.

Note that, like many works, the transmission delays of the network state information and control information bring very low costs and can be ignored since they are usually very tiny. In addition, these information can also be transmitted along with the task offloading process as protocol fields in the task data, or be directly obtained from the network state information service running in the network management system. In these cases, the relevant costs can be eliminated.

### E. Algorithm Complexity

The complexity of our algorithm depends on 1) the number of neurons in each layer of the trained primary actor network; 2) the complexity of the computing resource allocation sub-problem; 3) the complexity of the transmit power allocation sub-problem.

We assume the trained primary actor network contains $X$ fully connected layers and $Y_i$ is the number of neurons of the $i$-th layer. $Y_1$ and $Y_X$ are the input and output sizes of the network, respectively. Thus, the complexity of the network is $O\left(\sum_{i=1}^{X-1}(Y_iY_{i+1})\right)$.

Based on **Algorithm 1**, the time complexity of the computing resource allocation sub-problem of a single vehicle is

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

$O(2)$. Considering the worst case that all the vehicles process their tasks locally, the time complexity of the computing resource allocation of all the vehicles is $O(2\sum_{j\in\mathcal{M}} N_j)$.

Considering the worst case that all the vehicles offload their tasks, we denote by $W_{ij}$ the number of iterations required for the convergence of the Dinkelbach algorithm when it solves for vehicle $ij$. We can give the complexity of the transmit power control sub-problem as $O\big(\sum_{j\in\mathcal{M}}\sum_{i\in\mathcal{N}_j} W_{ij}\big)$. In experiments, the range of $W_{ij}$ is between 1 to 4, and its average value is only 2.81.

Thus, the total complexity of our algorithm in the worse case is $O\big(\sum_{i=1}^{X-1}(Y_i Y_{i+1}) + 2\sum_{j\in\mathcal{M}} N_j + \sum_{j\in\mathcal{M}}\sum_{i\in\mathcal{N}_j} W_{ij}\big)$, which is low and thus can demonstrate the efficient online performance of our algorithm.

## V. Experiment Results an Evaluations

### A. Experiment Settings

To evaluate the performance of our proposed scheme, we have conducted a large number of experiments. Our proposed TD3-based DRL algorithm is developed on the PyTorch platform (V1.12.1). The hyper parameters of the neural networks in our TD3 model are selected based on multiple comparative experiments, where we set different numbers of neurons in each layer of the networks, learning rates, activation functions, discounting factors, softupdate weights, update frequencies, numbers of episodes and steps, replay buffer sizes, etc. We chose the hyper parameter setting that makes the TD3 model have the highest reward after convergence. The actor network (primary and target) is a two-hidden-layer (2048,1024) fully-connected neural network; the critic network (primary and target) is also a two-hidden-layer (2048,1024) fully-connected neural network. We use Relu as the activation function in the hidden layer. To deal with the constraints of binary variables such as $C_1$ and $C_2$, the neurons in the output layer of the actor are activated by the $softmax$ function, while the neurons in the other layers of the actor and all the layers of the critic are activated by the $tanh$ function. The learning rates for the primary actor and critic networks are $\phi^{\pi} = 3\times10^{-5}$ and $\phi^{Q} = 3\times10^{-4}$, respectively. The discounting factor is $\xi = 0.99$. The softupdate weight is $\lambda = 0.995$. The optimizers of all the networks are set to Adam. The update frequency is $\tau = 50$. The number of episodes is 500 and the number of steps per episode is 500. The replay buffer size is $|Z| = 50000$ and the mini-batch size is $E = 200$. All the default parameter settings used in the simulations are illustrated in Table III. The value selections of the parameters are based on [2], [3], and [5], shown in the table.

We use a dataset of vehicle movement trajectories [34] to conduct experiments. The dataset includes the vehicle movement data of over 22,000 vehicles in the morning of a weekday in Bologna, Italy from 8.00 a.m. to 9.00 a.m. The data is sampled at an interval of 1s. We simulate high-load scenarios, normal-load scenarios, and low-load scenarios according to different congestion conditions from the corresponding regions in the dataset. Based on the latitude, longitude, and velocity of the vehicles at each time interval from the dataset, we can obtain the trajectory of each vehicle, and then calculate $\xi_{ij}$ and $\chi_{ij}$ of each vehicle $ij, i\in\mathcal{N}_j, j\in\mathcal{M}$ in its trajectory.

TABLE III
Parameters Used in Simulations and Their Default Values

| Parameter | Value | Parameter | Parameter |
|---|---|---|---|
| $M$ | 5 | $F_{ij}^{v}$ | $5\sim6$ G CPU cycles/s [2] |
| $N_j$ | $10\sim50$ | $F_{k}^{r}$ | $30\sim50$ G CPU cycles/s [2] |
| $d_{ij}^{in}$ | $1\sim10$ MB [5] | $P_{ij}^{v2r}$ | 2 W [2] |
| $d_{ij}^{out}$ | $0.01\sim20$ MB [5] | $N_0$ | $1\times10^{-13}$ W/Hz [2] |
| $c_{ij}$ | $2\sim8$ G CPU cycles [2] | $H_{ij}^{v2r}, H_{ijl}^{r2v}$ | $10^{-5}$ [5] |
| $W$ | 15MHz [5] | $I_{ij}^{v2r}, I_{ijl}^{r2v}$ | $10^{-13}$ W [3] |
| $R_j^{r2r}$ | $1\sim2$ G bit/s [3] | $\kappa_{ij}$ | $3\sim5\times10^{-27}$ [5] |

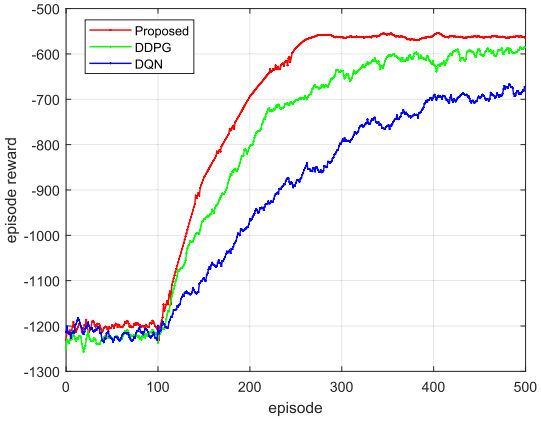



Fig. 2. Convergence of the episode reward of the proposed scheme.

### B. Convergence Analysis

In Fig. 2, we evaluate the convergence of the proposed algorithm with respect to the training episodes. As shown in Fig. 2, the curve in the figure represents the episode reward. It can be seen that after the replay buffer is full, the episode reward increases with the increase of the training episode, and then gradually becomes stable and convergent at around the $280th$ training episode. The average episode reward is -546.6 for the last 100 episodes.

We also conducted a comparative study of the performance of TD3, DDPG, and DQN algorithms. The experimental results showed that the DDPG algorithm gradually converged and stabilized around the 420th training iteration, while the DQN algorithm did not converge within the first 500 training iterations. It can be observed that TD3 achieved significantly higher total rewards compared to the two traditional methods, and its curve was the smoothest. This improved performance can be attributed to the combined use of techniques such as target policy network, twin Q-networks, and delayed updates in TD3. These techniques effectively reduce noise and instability during the training process, allowing the algorithm to converge faster and achieve better results.

### C. Comparative Schemes And Performance Evaluation

The performance of our scheme is compared with 4 other schemes:

1) Task Result Transmission Only ($TRTO$): This scheme does not support load transfer among RSUs because R2R task offloading is disabled. The tasks of the vehicles can be only

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

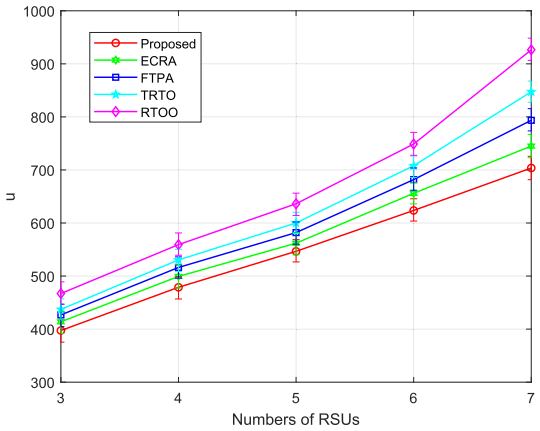



Fig. 3. $u$ vs. numbers of RSUs.

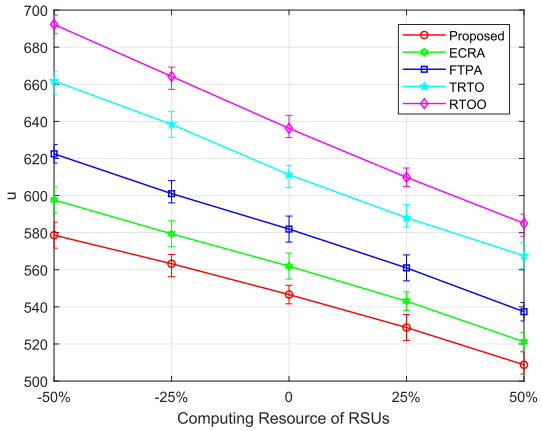



Fig. 4. $u$ vs. computing resources of RSUs.

offloaded to their original RSUs and the task results can be transmitted to the RSUs that currently cover the vehicles. This scheme embodies the ideas of the works in [4] and [5].

2) R2R Task Offloading Only ($RTOO$): This scheme does not support vehicle mobility because the task results of the vehicles have to be transmitted to their original RSUs. R2R task offloading is enabled, so a task can be further offloaded from the original RSU to another RSU. If a vehicle will move out of the coverage of its original RSU before receiving the task result, the vehicle can only choose to process its task locally. This scheme embodies the ideas of the works in [6], [7], and [8].

3) Equal Computing Resource Allocation ($ECRA$): In the scheme, each RSU allocates computing resources equally to the tasks offloaded to it. The rest of the program is the same as our scheme.

4) Fixed Transmit Power Allocation ($FTPA$): In the scheme, each vehicle uses the maximum value of its transmitted power for task offloading. The rest of the program is the same as our scheme.

In the following part, we evaluate the performance of every scheme in 6 scenarios (i.e., we change 6 kinds of parameters from 50% to 150% of their default value), and then we analyze the results. The simulation results for each scenario are averaged from 50 independent simulations. In Fig. 3 - 5 and Fig. 7 - 9, the curves represent the average values and the labels on each curve indicate the maximum and minimum values in the independent simulations. In summary, compared with 4 reference schemes, our scheme can reduce the total task processing cost by 17.3%-28.4%.

1) *Numbers of RSUs*: As depicted in Fig. 3, we increase the number of RSUs from 3 to 7, while setting the other parameters as default. It can be seen that the performance of all the schemes declines. This is because as the size of the system increases, the number of tasks generated by the vehicles in the whole system becomes larger, and the corresponding latency and energy consumption increase. Among these schemes, our scheme always has the best performance under different numbers of RSUs. The difference between the 5 schemes is slight when the system size is small, and the difference gradually increases as the system scale increases. This is mainly because our scheme can reduce the load imbalance of the whole system by transferring the tasks on high-load RSUs to other low-load RSUs. This advantage becomes more significant as the number of RSUs increases, because both the computing resources of the whole system and the probability of vehicles moving among RSUs increase, so that flexible RSU-RSU cooperation can bring better results. The $TRTO$ and $RTOO$ constrain the flexible RSU-RSU cooperation, therefore, they cannot reduce the system load imbalance and solve the task result reception failure problem, leading to system performance degradation.

2) *Computing Resources of RSUs*: As depicted in Fig. 4, when we change the lower bound and upper bound of the computing resources of RSUs from 50% to 150% of its default value while setting other parameters as default, the performance of all the schemes increases. That is because the computing resources keep increasing, providing more abundant computing capabilities for the offloaded tasks, so the computing delay of the tasks on the RSUs decreases, and the system performance improves correspondingly. However, compared with the other 4 schemes, it can be seen that the advantage of our scheme shrinks since the enlargement of the computing resources alleviates the load pressures of the RSUs, thus, the performance improvement brought by the load transfer becomes less significant. As the computing resources of the RSUs increase, we can see that the gap between the $ECRA$ and our scheme becomes smaller because the $ECRA$ distributes the computing resources of the RSUs equally to all tasks for computation, and when the computing resources become abundant, there are enough computing capabilities for task computing, so the equal computing resource allocation does not have much impact on the system performance. The performance of the $RTOO$, $TRTO$, $ECRA$, and $FTPA$ is worse than that of our scheme owing to the lack of task result transmission, load transfer, computing resource allocation, and transmit power allocation, respectively.

3) *Numbers of High-Load RSUs*: As depicted in Fig. 5, we add extra high-load RSUs to the scenario, while setting other parameters as default. The number of high-load RSUs (50 vehicles covered by each high-load RSU) increases from 0 to 4, so the total number of RSUs is increased from 5 to 9. In this case, the performance of all the 5 schemes declines. This is because as the number of vehicles increases and the number of tasks increases, the corresponding system

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

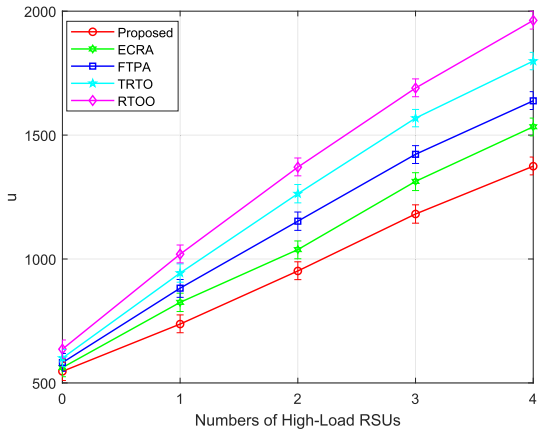



Fig. 5. $u$ vs. numbers of high-load RSUs.

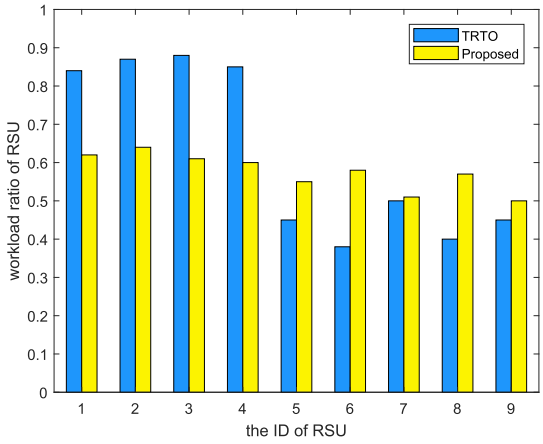



Fig. 6. RSU workloads with and without load transfer.

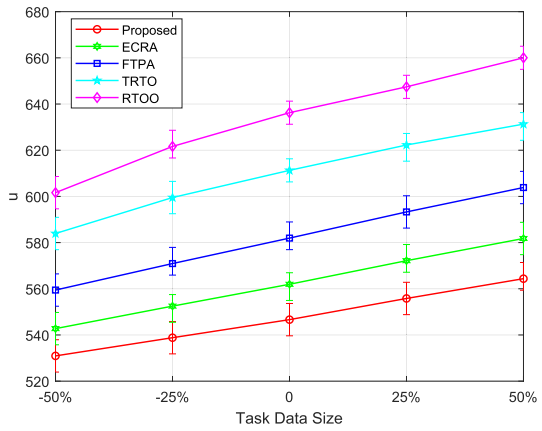



Fig. 7. $u$ vs. task data size.

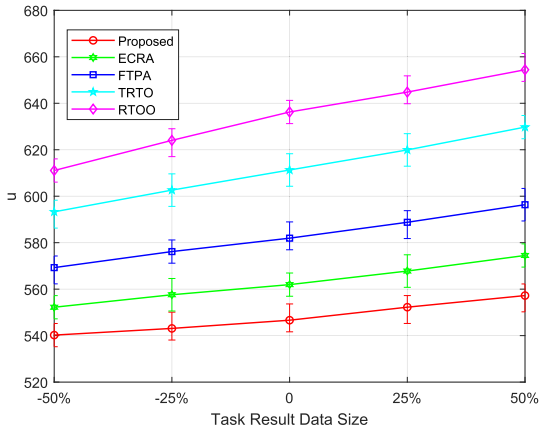



Fig. 8. $u$ vs. task result data size.

latency and energy consumption increases. Our scheme always has the best performance compared with the $TRTO$ and $RTOO$, mainly because it utilizes flexible RSU-RSU cooperation to reduce the load imbalance among RSU. Therefore, the load pressures of the high-load RSUs can be shared with the other RSUs to improve system performance, especially when the number of high-load RSUs is high.

To further explain the effect of load transfer, in Fig. 6, we show the RSU workloads with load transfer (i.e. adopting our scheme) and without load transfer (i.e. adopting the $TRTO$) in a scenario consisting of 4 high-load RSUs (RSU 1-4) and 5 other RSUs (RSU 5-9). We define the load of each RSU as the ratio of the computation of all the tasks offloaded to the RSU to the total computing resources of the RSU. We can see that the loads of RSU 1-4 are much higher than the loads of RSU 5-9 before enabling load transfer, and the difference between RSU 1-4 and RSU 5-9 decreases after enabling load transfer.

4) *Task Data Size*: As depicted in Fig. 7, we change the lower bound and upper bound of the task data size from 50% to 150% of its default value, while setting other parameters as default. It can be found that the performance of all the schemes declines, because both the V2R task transmission delay and the R2R task transmission delay are prolonged by the increase of the task data size, making the total task processing delay increase. Also, as the V2R task transmission delay increases, the energy consumption rises correspondingly, leading to a decline in the total task processing cost. Our scheme can always obtain the lowest total task processing cost, and the advantage becomes higher as the system load increases. The main reason is that the load transfer via flexible RSU-RSU cooperation plays an important role in improving the system performance.

5) *Task Result Data Size*: As depicted in Fig. 8, when we change the lower bound and upper bound of the task result size of vehicle task from 50% to 150% of its default value, the performance of all the 5 schemes declines. The reason is that when the R2V task result transmission delay increases as the task result size increases, the total task processing delay increases, and then the system performance decreases correspondingly. In the scheme $TRTO$, only the task result transfer exists, and the task result transfer cannot be reduced by the task transfer, so the increase of task result data size causes the increase of R2R transfer delay, which leads to a significant degradation of system performance. In scheme $RTOO$, it is required that task results must be transferred back to the initial RSU, otherwise local processing, task result data size increases, R2R transfer latency increases, and system performance decreases. We can see that the performance of our proposed scheme is superior to the other 4 schemes. The advantage becomes larger as the task result size increases.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

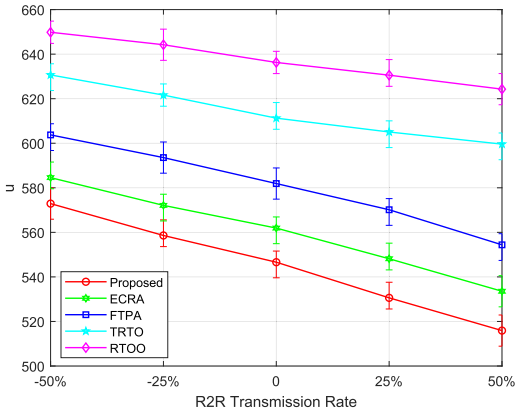



Fig. 9. $u$ vs. R2R transmission rates.

6) *R2R Transmission Rates*: As depicted in Fig. 9, we change the lower bound and upper bound of the R2R transmission rates of the RSUs from 50% to 150% of its default value, while setting other parameters as default. We can observe that the performance of all the 5 schemes increases, but the performance of the $TRTO$ and $RTOO$ only improves to a less extent. That is because the delays in transmitting the tasks and task results among RSUs decrease, leading to a decrease in the overall task processing delay. The increasing R2R transmission rates also make the task offloading strategy tend to further offload the tasks on high-load RSUs to low-load RSUs owing to smaller R2R transmission delay, thus reducing the total task processing delay. The $TRTO$ involves the R2R task result transmission only, while the $RTOO$ involves R2R task transmission only, so the change in R2R transmission rates does not affect the two schemes as much as the other 3 schemes. We can see that the performance of our proposed scheme is superior to the other 4 schemes.

From Figs. 3-9, it can be concluded that the performance of all the schemes is our scheme $>$ $ECRO$ $>$ $FTPA$ $>$ $TRTO > RTOO$.

## VI. Conclusion

In this paper, we propose a task offloading and resource allocation scheme for multi-vehicle multi-RSU VEC scenarios. To avoid task result reception failures and load imbalance among RSUs, we exploit flexible RSU-RSU cooperation to enable both task offloading and task result transmission among RSUs. An optimization problem is formulated to minimize the total task processing delay and energy consumption of all the vehicles by jointly scheduling the task offloading requests and task results, allocating the computing resources of all the vehicles and RSUs, allocating the V2R transmit powers of all the vehicles, and allocating the R2R transmission rates of all the RSUs. A TD3-based DRL algorithm is proposed to solve the problem efficiently. The algorithm is embedded with an optimization subroutine to solve the computing resource allocation and transmit power allocation sub-problems of the vehicle by using numerical methods. The convergence of the algorithm is evaluated. Extensive simulations are conducted in 6 scenarios by varying crucial parameters. The superiority of our scheme is fully demonstrated in comparison with 4 other reference schemes.

In the future, we plan to further investigate the resource management of VEC in multi-service scenarios, where the vehicles have different tasks corresponding to different services with diverse QoS requirements. We plan to add service placement and QoS guarantee into the optimization and design high-efficiency algorithms.

## References

[1] W. Fan et al., "Joint task offloading and resource allocation for vehicular edge computing based on V2I and V2V modes," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4277–4292, Apr. 2023.

[2] W. Fan, J. Liu, M. Hua, F. Wu, and Y. Liu, "Joint task offloading and resource allocation for multi-access edge computing assisted by parked and moving vehicles," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5314–5330, May 2022.

[3] W. Fan et al., "Game-based task offloading and resource allocation for vehicular edge computing with edge-edge cooperation," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7857–7870, Jun. 2023.

[4] K. Zhang, Y. Mao, S. Leng, Y. He, and Y. Zhang, "Mobile-edge computing for vehicular networks: A promising network paradigm with predictive off-loading," *IEEE Veh. Technol. Mag.*, vol. 12, no. 2, pp. 36–44, Jun. 2017.

[5] H. Wang, T. Lv, Z. Lin, and J. Zeng, "Energy-delay minimization of task migration based on game theory in MEC-assisted vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 8175–8188, Aug. 2022.

[6] D. Pliatsios, P. Sarigiannidis, T. D. Lagkas, V. Argyriou, A. A. Boulogeorgos, and P. Baziana, "Joint wireless resource and computation offloading optimization for energy efficient Internet of Vehicles," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 3, pp. 1468–1480, Sep. 2022.

[7] Z. Xiao et al., "Vehicular task offloading via heat-aware MEC cooperation using game-theoretic method," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 2038–2052, Mar. 2020.

[8] J. Zhang, H. Guo, J. Liu, and Y. Zhang, "Task offloading in vehicular edge computing networks: A load-balancing solution," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2092–2104, Feb. 2020.

[9] K. Zhang, Y. Mao, S. Leng, S. Maharjan, and Y. Zhang, "Optimal delay constrained offloading for vehicular edge computing networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2017, pp. 1–6.

[10] Z. Wang, S. Zheng, Q. Ge, and K. Li, "Online offloading scheduling and resource allocation algorithms for vehicular edge computing system," *IEEE Access*, vol. 8, pp. 52428–52442, 2020.

[11] S. Pang, N. Wang, M. Wang, S. Qiao, X. Zhai, and N. N. Xiong, "A smart network resource management system for high mobility edge computing in 5G Internet of Vehicles," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 4, pp. 3179–3191, Oct. 2021.

[12] C. Ren, G. Zhang, X. Gu, and Y. Li, "Computing offloading in vehicular edge computing networks: Full or partial offloading?" in *Proc. IEEE 6th Inf. Technol. Mechatronics Eng. Conf. (ITOEC)*, vol. 6, Mar. 2022, pp. 693–698.

[13] M. Liwang, S. Dai, Z. Gao, Y. Tang, and H. Dai, "A truthful reverse-auction mechanism for computation offloading in cloud-enabled vehicular network," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4214–4227, Jun. 2019.

[14] B. L. Nguyen, D. T. Ngo, N. H. Tran, M. N. Dao, and H. L. Vu, "Dynamic V2I/V2V cooperative scheme for connectivity and throughput enhancement," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 1236–1246, Feb. 2022.

[15] L. Zhang, Y. Sun, Y. Tang, H. Zeng, and Y. Ruan, "Joint offloading decision and resource allocation in MEC-enabled vehicular networks," in *Proc. IEEE 93rd Veh. Technol. Conf. (VTC-Spring)*, Apr. 2021, pp. 1–5.

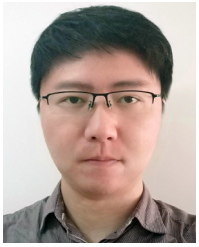
[16] X. Zhu, Y. Luo, A. Liu, M. Z. A. Bhuiyan, and S. Zhang, "Multiagent deep reinforcement learning for vehicular computation offloading in IoT," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9763–9773, Jun. 2021.

[17] X. Xu, Z. Liu, M. Bilal, S. Vimal, and H. Song, "Computation offloading and service caching for intelligent transportation systems with digital twin," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20757–20772, Nov. 2022.

[18] X. Zhou et al., "Edge computation offloading with content caching in 6G-enabled IoV," *IEEE Trans. Intell. Transp. Syst.*, early access, 2023, doi: 10.1109/TITS.2023.3239599.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.

[19] T. Bahreini, M. Brocanelli, and D. Grosu, "VECMAN: A framework for energy-aware resource management in vehicular edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 22, no. 2, pp. 1231–1245, Feb. 2023.
[20] H. Cho, Y. Cui, and J. Lee, "Energy-efficient cooperative offloading for edge computing-enabled vehicular networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 10709–10723, Dec. 2022.
[21] X. Huang, L. He, X. Chen, L. Wang, and F. Li, "Revenue and energy efficiency-driven delay-constrained computing task offloading and resource allocation in a vehicular edge computing network: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8852–8868, Jun. 2022.
[22] M. Liwang, J. Wang, Z. Gao, X. Du, and M. Guizani, "Game theory based opportunistic computation offloading in cloud-enabled IoV," *IEEE Access*, vol. 7, pp. 32551–32561, 2019.
[23] X. Hou et al., "Reliable computation offloading for edge-computing-enabled software-defined IoV," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020.
[24] D. Zhu, M. Bilal, and X. Xu, "Edge task migration with 6G-enabled network in box for cybertwin-based Internet of Vehicles," *IEEE Trans. Ind. Informat.*, vol. 18, no. 7, pp. 4893–4901, Jul. 2022.
[25] L. Yao, X. Xu, M. Bilal, and H. Wang, "Dynamic edge computation offloading for Internet of Vehicles with deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 12991–12999, Nov. 2023.
[26] Z. Wei, B. Li, R. Zhang, X. Cheng, and L. Yang, "Many-to-many task offloading in vehicular fog computing: A multi-agent deep reinforcement learning approach," *IEEE Trans. Mobile Comput.*, early access, 2023, doi: 10.1109/TMC.2023.3250495.
[27] P. Liu, J. Li, and Z. Sun, "Matching-based task offloading for vehicular edge computing," *IEEE Access*, vol. 7, pp. 27628–27640, 2019.
[28] Y. Hui, Z. Su, T. H. Luan, and J. Cai, "Content in motion: An edge computing based relay scheme for content dissemination in urban vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 8, pp. 3115–3128, Aug. 2019.
[29] J. Shi, J. Du, J. Wang, and J. Yuan, "Distributed V2V computation offloading based on dynamic pricing using deep reinforcement learning," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, May 2020, pp. 1–6.
[30] X. Huang, L. He, and W. Zhang, "Vehicle speed aware computing task offloading and resource allocation based on multi-agent reinforcement learning in a vehicular edge computing network," in *Proc. IEEE Int. Conf. Edge Comput. (EDGE)*, Oct. 2020, pp. 1–8.
[31] Y. Wen, W. Zhang, and H. Luo, "Energy-optimal mobile application execution: Taming resource-poor mobile devices with cloud clones," in *Proc. IEEE INFOCOM*, Mar. 2012, pp. 2716–2720.
[32] W. Dinkelbach, "On nonlinear fractional programming," *Manage. Sci.*, vol. 13, no. 7, pp. 492–498, Mar. 1967.
[33] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
[34] L. Bedogni, M. Gramaglia, A. Vesco, M. Fiore, J. Härri, and F. Ferrero, "The Bologna ringway dataset: Improving road network conversion in SUMO and validating urban mobility via navigation services," *IEEE Trans. Veh. Technol.*, vol. 64, no. 12, pp. 5464–5476, Dec. 2015.



**Wenhao Fan** (Member, IEEE) received the B.E. and Ph.D. degrees from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2008 and 2013, respectively.

He is currently an Associate Professor (the Supervisor of master's and doctoral students) with the School of Electronic Engineering, BUPT. He has authored or coauthored nearly 100 academic articles and received over 40 patents for inventions. His main research topics include communication networks, edge/cloud computing, edge intelligence, the IoT, network security, and AI-empowered networking technologies. He has won the Second Prize in the 2021 Chinese Institute of Electronics (CIE) Scientific and Technical Progress Reward; the Second Prize in the 2021 Chinese Association for Artificial Intelligence (CAAI) Technology Invention Award; the 2018 Excellent Scientific and Technological Achievement Award of Guangdong Province, China; and the First Prize in the 2017 CIE Scientific and Technical Progress Reward.



**Yaoyin Zhang** received the B.E. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2021. She is currently a Postgraduate Student with the School of Electronic Engineering, BUPT.

Her main research topics include edge computing and the Internet of Vehicles.



**Guangtao Zhou** received the Ph.D. degree in engineering.

He is a senior engineer with the professorial level. He is an Intelligent Connection Scientist and the Director of the Vehicles Intelligent Connection Research Institute, China Unicom Smart Connection Technology Company Ltd., Beijing, China. Additionally, he is honored to receive a special allowance from the State Council. He has over 15 years of experience in telecommunications industry, and worked for various telecom operators. As a technical expert, he has actively participated in international research on technical standards and product innovation for a long time. His main research topics include the innovative applications of 5G/V2X-based autonomous driving and intelligent transportation.



**Yuan'an Liu** received the B.E., M.Eng., and Ph.D. degrees in electrical engineering from the University of Electronic Science and Technology, Chengdu, China, in 1984, 1989, and 1992, respectively.

He is currently a Professor with the School of Electronic Engineering, Beijing University of Posts and Telecommunications (BUPT). He was the Dean of the Beijing Key Laboratory of Work Safety Intelligent Monitoring, BUPT. His main research topics include wireless communications, electromagnetic compatibility, and pervasive computing. He is a fellow of the Institution of Engineering and Technology, U.K., and the Electronic Institute of China; the Vice Chairperson of electromagnetic environment and safety with the China Communication Standards Association; and the Vice Director of the Wireless and Mobile Communication Committee, Communication Institute of China.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:07:16 UTC from IEEE Xplore. Restrictions apply.